Architecture Goals & Constraints

Non-Functional Requirement	What	Why	How
Reliability	1. The system should correctly map story structures to projected chapters. 2. The system should correctly calculate the projected number of chapters based on the target word count and average chapter length.	Users rely on the structure mapping feature to plan their project. If beats/acts map to the wrong chapters, or the chapter count is wrong, the writing plan becomes misleading and undermines trust in the tool.	• Cover the percentage-to-chapter mapping algorithm with unit tests across edge cases (few chapters, overlapping percentages, 0%/100% boundaries) • Validate structure input (percentages within 0–100, required fields) before mapping • Validate that word count and chapter length are positive numbers before calculating chapter count
Performance	1. The system should provide fast response times and smooth UI interactions. 2. The system should have a short start-up time.	The structure editor involves frequent, incremental interactions (adding/reordering acts, beats, scenes); noticeable lag disrupts the creative workflow.	• Avoid recomputing the full structure-to-chapter mapping on every keystroke; recompute only when relevant inputs change • Keep state updates local to the component being edited, rather than re-rendering the whole tree
Supportability	1. 1. The core logic, such as chapter calculation and story structure mapping, should be testable independently of the UI. 2. 2. The codebase should be easy to maintain and extend, such as when adding new built-in story structures.	The mapping algorithm is the core value of the app; bugs here are hard to catch visually and need to be caught through tests. New features (e.g. new structures) should be addable without breaking existing behaviour.	• Write pure, UI-independent functions for domain logic and cover them with unit tests • Keep UI, domain logic, and persistence layers separated, as reflected in the architecture diagram
Usability	1. The system should provide an aesthetically pleasing and consistent user interface. 2. The system should be easy to use. 3. Instructions for the story structure customisation feature should be clear and easy to follow.	Inconsistent styling across views (project dashboard, chapter planner, structure editor) makes the app feel unpolished. Aesthetics would be a greatly valued by our target users(authors). The custom structure editor (acts/beats/ with percentages) should be easy to follow.	• Define and apply a shared design system (colour, spacing, styles) across all pages • Keep core workflows (create project, plan chapters) simple by default; keep advanced features (custom structure) optional • Provide inline guidance, example templates, and a preview of how beats map to chapters before saving
Compatibility	1. The system should work on the devices and platforms agreed with client, including supported operating systems and web browsers, such as Chrome. 2. Exported documents should	To prevent compatibility problems caused by differences in devices, operating system versions, browsers, and document formats.	• Define the supported platforms and versions, test the system in each supported environment, and test every supported document format.

	be able to be opened using the user's preferred tools, such as Excel.		
Maintainability	1. Code and documentation should be clear and well organized so that team members can work together effectively. 2. Current and future developers should be able to understand the project, fix problems, and add new features.	Team may need to respond to client feedback and fix problems after each review. After handover, the client may also ask other developers to maintain or extend the system.	• Organize the project artefacts into clearly named folders for source code, requirements, instructions, and tests. Use consistent naming and keep all files up to date.
Privacy	1. The system should minimise unnecessary transmission of users' story planning data to external services.	Manuscript planning information may be private and users may prefer to keep it on their own device.	• Store project data locally using IndexedDB rather than transmitting project content to a backend database.
Extensibility	1. The system should support the addition of new planning features and story structures in the future.	To allow the application to evolve as additional authoring requirements are identified.	• Use reusable and loosely coupled components and a flexible data model for story structures and story beats.
Data Portability	1. The system should allow users to transfer and back up their project data.	To allow users to preserve their work and move project data when required	• Provide JSON export and import functionality.
Data Persistence	1. The system should ensure that project data is saved locally and remains available across browser sessions.	To prevent users from losing their story planning data when they close or refresh the application.	• Store project, chapter, character, event and story structure data in IndexedDB